

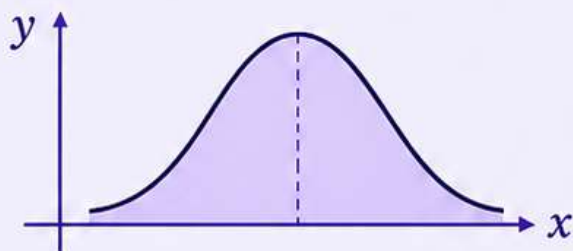
01

PHASE

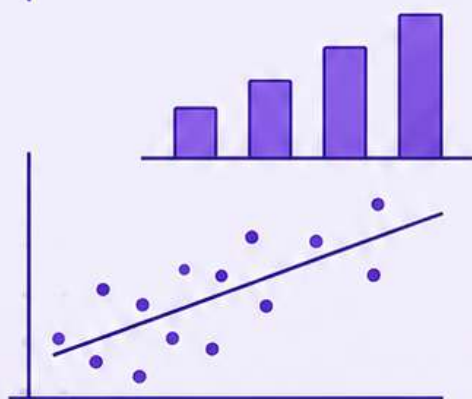
Mathematics & Statistics for Data Analyst

Build Strong Foundations.
Solve Real-World Problems.

$$\sum_{i=1}^n x_i$$



$$\sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2}$$



$$X = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{bmatrix}$$



Mathematical
Concepts



Statistical
Methods



Data Analysis
Techniques



Problem Solving
Skills



Master the Basics. Excel in Analytics.

UNIT 1

Concise, Comprehensive & Easy-to-Understand

What's Inside

1.1 Measures of center

- i) Mean, Median & Mode
- ii) Trimmed & Winsorized Mean
- iii) Midrange & Mid-quartile

1.2 Measures of spread

- i) Range & Interquartile Range
- ii) Variance & Standard Deviation
- iii) Mean Absolute Deviation (MAD)
- iv) Coefficient of Variation (CV)

1.3 Shape & position

- i) Skewness (Pearson & Fisher)
- ii) Kurtosis & Excess Kurtosis
- iii) Percentiles, Quartiles & Deciles
- iv) Z-scores & Standardisation

1.4 Visualisation

- i) Histograms & Bin Width Selection
- ii) Box plots & Violin plots
- iii) Stem-and-leaf & QQ plots
- iv) Frequency & Contingency Tables

UNIT 1 — Descriptive Statistics | Summarizing & Describing Data

Descriptive statistics are tools that help us summarize, describe, and understand the key features of a dataset — without making predictions or inferences about a larger population. Think of it as taking a snapshot of your data.

1.1 Measures of Center

Measures of center identify the 'typical' or 'central' value in a dataset — the single number that best represents where most data points cluster.

i) Mean, Median & Mode

DEFINITION

Mean: The arithmetic average — sum of all values divided by the count.

Median: The middle value when data is sorted; splits data into two equal halves.

Mode: The value(s) that appear most frequently in the dataset.

SUB-TOPICS

- Arithmetic Mean: Standard average ($\sum x / n$).
- Weighted Mean: Each value carries a different importance/weight.
- Geometric Mean: n th root of the product of n values — used for growth rates.
- Harmonic Mean: Reciprocal of the average of reciprocals — used for rates/speeds.
- Unimodal / Bimodal / Multimodal: Datasets with one, two, or many modes.

WHY IMPORTANT

These three measures form the foundation of any data summary. Choosing the right one matters: the mean is sensitive to outliers, the median is robust, and the mode reveals the most common occurrence. Together they describe the 'center of gravity' of a distribution.

REAL-WORLD ANALOGY

Imagine 5 friends earning ₹20k, ₹22k, ₹25k, ₹28k, and ₹200k per month. The mean salary (~₹59k) is skewed by the high earner. The median (₹25k) is far more representative. Mode = no single value repeats, so mode is less useful here.

SYNTAX (Python & Excel)

```
# Python (NumPy / Statistics)
import numpy as np, statistics as st
data = [20000, 22000, 25000, 28000, 200000]

mean = np.mean(data) # 59000.0
median = np.median(data) # 25000.0
mode = st.mode(data) # no single mode
```

```
# Excel equivalents
# =AVERAGE(A1:A5) → Mean
# =MEDIAN(A1:A5) → Median
# =MODE(A1:A5) → Mode
```

EXAMPLE

Dataset: [4, 7, 7, 9, 10, 15]

Mean = $(4+7+7+9+10+15) / 6 = 52/6 \approx 8.67$

Median = $(7+9)/2 = 8.0$ (average of 3rd & 4th values)

Mode = 7 (appears twice)

INDUSTRY USE CASE

- E-commerce: Mean order value tracks overall revenue health; median order value is used when a few large orders distort the picture.
- Healthcare: Modal blood pressure reading identifies the most typical patient reading in a clinic.
- HR / Payroll: Median salary is reported in job postings to give fairer pay benchmarks.

ii) Trimmed Mean & Winsorized Mean

DEFINITION

Trimmed Mean: Remove a fixed percentage of the lowest and highest values, then compute the mean on the remaining data. This reduces outlier influence.

Winsorized Mean: Instead of removing extremes, replace them with the nearest boundary value (e.g., replace the bottom 5% with the 5th percentile value), then take the mean.

SUB-TOPICS

- 10% Trimmed Mean: Drop bottom 10% and top 10% of values, compute mean on remaining 80%.
- Winsorization at k%: Clamp values below the k-th percentile to that percentile value, and similarly for the top.

WHY IMPORTANT

Real-world data often contains outliers (errors, rare events, fraud). The ordinary mean breaks down in such cases. Trimmed and Winsorized means give a robust central estimate without being pulled by extreme values, while keeping more information than the median.

REAL-WORLD ANALOGY

In Olympic figure skating, judges drop the highest and lowest scores before averaging — that's a trimmed mean in action!

SYNTAX (Python)

```
from scipy.stats import trim_mean, mstats
import numpy as np

data = [2, 4, 5, 6, 7, 8, 9, 100]

# 10% Trimmed Mean (drops 10% from each end)
trimmed = trim_mean(data, proportiontocut=0.10)

# Winsorized Mean (clamp at 10th & 90th percentile)
winsorized_data = mstats.winsorize(data, limits=[0.10, 0.10])
win_mean = np.mean(winsorized_data)
```

EXAMPLE

Data: [2, 4, 5, 6, 7, 8, 9, 100] (100 is an outlier)

Regular Mean = 17.6 ← heavily skewed

Trimmed Mean (10%) → drop 2 and 100 → mean([4,5,6,7,8,9]) = 6.5

Winsorized Mean (10%) → replace 100 with 9 → mean([2,4,5,6,7,8,9,9]) = 6.25

INDUSTRY USE CASE

- Finance: Trimmed returns are used in performance reporting to avoid distortion by extreme trading days.
- Survey Research: Winsorized means handle scale responses distorted by troll or random answers.

iii) Midrange & Mid-quartile

DEFINITION

Midrange: The average of the minimum and maximum values — the centre of the data range.

$$\text{Midrange} = (\text{Min} + \text{Max}) / 2$$

Mid-quartile (Midhinge): The average of Q1 (25th percentile) and Q3 (75th percentile).

$$\text{Mid-quartile} = (Q1 + Q3) / 2$$

WHY IMPORTANT

Midrange is the simplest center estimate — fast to compute but very sensitive to outliers. Mid-quartile is more robust, capturing the center of the 'middle half' of the data and complementing the median.

REAL-WORLD ANALOGY

Midrange: If a shop's cheapest item is ₹10 and priciest is ₹990, the midrange price is ₹500. But if most items cluster around ₹50, the midrange is misleading.

Mid-quartile focuses on where the 'middle 50%' of prices sit — far more useful.

SYNTAX (Python)

```
import numpy as np

data = [10, 20, 25, 30, 35, 40, 50, 990]

midrange = (np.min(data) + np.max(data)) / 2 # 500.0
q1, q3 = np.percentile(data, [25, 75])
mid_quartile = (q1 + q3) / 2 # 32.5
```

INDUSTRY USE CASE

- Retail Pricing: Mid-quartile helps set 'fair price' benchmarks unaffected by luxury or clearance outliers.
- Weather / Climate: Midrange temperature gives a quick summary; mid-quartile better represents typical day temperatures.

1.2 Measures of Spread (Dispersion)

Measures of spread quantify how much data values vary or deviate from the center. Two datasets can have the same mean but very different spreads — understanding spread is critical for assessing risk, consistency, and reliability.

i) Range & Interquartile Range (IQR)

DEFINITION

Range: The simplest spread measure — the difference between the maximum and minimum.

Range = Max – Min

Interquartile Range (IQR): The range of the middle 50% of data.

IQR = Q3 – Q1 (75th percentile minus 25th percentile)

SUB-TOPICS

- Outlier Detection via IQR: Values below $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$ are flagged as outliers.
- Interdecile Range: Range between 10th and 90th percentile — a broader robust range.

WHY IMPORTANT

Range gives an instant feel for data span but is fragile — one extreme outlier inflates it dramatically. IQR is much more robust and is the backbone of box plots and outlier detection rules.

REAL-WORLD ANALOGY

Imagine exam scores: 10 students score between 40–95. Range = 55. But IQR tells you the spread of the 'typical' middle students — ignoring the weakest and strongest performers.

SYNTAX (Python)

```
import numpy as np
from scipy.stats import iqr

data = [40, 55, 60, 65, 70, 72, 75, 80, 88, 95]

range_val = np.max(data) - np.min(data) # 55
iqr_val = iqr(data) # IQR = Q3 - Q1

q1, q3 = np.percentile(data, [25, 75])
lower_fence = q1 - 1.5 * iqr_val # Outlier lower bound
upper_fence = q3 + 1.5 * iqr_val # Outlier upper bound
```

EXAMPLE

Scores: [40, 55, 60, 65, 70, 72, 75, 80, 88, 95]

Q1 = 61.25, Q3 = 79.0

IQR = 79.0 – 61.25 = 17.75

Lower fence = 61.25 – 1.5×17.75 = 34.625

Upper fence = 79.0 + 1.5×17.75 = 105.625

→ No outliers in this dataset.

INDUSTRY USE CASE

- Manufacturing Quality Control: IQR monitors product dimension variability; parts outside fences are flagged for rework.
- Cybersecurity: IQR-based outlier detection flags unusually large data transfers as potential breaches.

ii) Variance & Standard Deviation

DEFINITION

Variance: The average of the squared differences from the mean. It measures how spread out values are.

Population Variance (σ^2) = $\Sigma(x_i - \mu)^2 / N$

Sample Variance (s^2) = $\sum(x_i - \bar{x})^2 / (n-1)$ ← Bessel's correction

Standard Deviation: The square root of variance, expressed in the same units as the original data.

$$\sigma = \sqrt{\sigma^2} \text{ (population)} \quad s = \sqrt{s^2} \text{ (sample)}$$

SUB-TOPICS

- Population vs Sample: Use N for entire population, n-1 for a sample (Bessel's correction removes bias).
- Why n-1?: A sample underestimates spread; subtracting 1 degree of freedom corrects this.
- Pooled Variance: Combines variance from two groups — used in t-tests and ANOVA.

WHY IMPORTANT

Standard deviation is the most widely used spread measure in statistics, machine learning, and finance. It forms the basis of z-scores, confidence intervals, control charts, and the normal distribution.

REAL-WORLD ANALOGY

Two factories produce bolts of 10mm length. Factory A's bolts range from 9.9–10.1mm (low SD). Factory B's range from 8–12mm (high SD). Even though both average 10mm, Factory A is far more consistent.

SYNTAX (Python)

```
import numpy as np

data = [4, 7, 13, 2, 1]

pop_var = np.var(data, ddof=0) # Population variance (ddof=0)
samp_var = np.var(data, ddof=1) # Sample variance (ddof=1)
pop_std = np.std(data, ddof=0) # Population std dev
samp_std = np.std(data, ddof=1) # Sample std dev

# Excel: =VAR(A1:A5) → Sample Variance
# =STDEV(A1:A5) → Sample Std Dev
```

EXAMPLE

Data: [2, 4, 4, 4, 5, 5, 7, 9] Mean = 5

Deviations from mean: [-3, -1, -1, -1, 0, 0, 2, 4]

Squared deviations: [9, 1, 1, 1, 0, 0, 4, 16]

Population Variance $\sigma^2 = (9+1+1+1+0+0+4+16) / 8 = 4.0$

Population Std Dev $\sigma = \sqrt{4} = 2.0$

INDUSTRY USE CASE

- Finance / Investment: Portfolio standard deviation measures risk. Higher SD = more volatile investment.
- Manufacturing: Six Sigma uses standard deviation to define process control limits ($\pm 3\sigma$).
- Machine Learning: Feature standardization uses mean and std dev (z-score normalization).

iii) Mean Absolute Deviation (MAD)

DEFINITION

MAD is the average of the absolute differences between each data point and the mean.

$$\text{MAD} = \sum |x_i - \bar{x}| / n$$

Note: Sometimes 'MAD' refers to Median Absolute Deviation (a more robust version):

$$\text{Median MAD} = \text{median}(|x_i - \text{median}(x)|)$$

WHY IMPORTANT

Unlike variance, MAD doesn't square differences, so large outliers have less exaggerated influence. It's more interpretable (same units as data) and more robust than standard deviation. Median MAD is extremely resistant to outliers.

REAL-WORLD ANALOGY

If your daily commute times are 30, 32, 28, 35, 31 minutes, MAD tells you how many minutes you typically deviate from your average commute — without letting one unusually long trip dominate the calculation.

SYNTAX (Python)

```
import numpy as np
from scipy.stats import median_abs_deviation

data = [30, 32, 28, 35, 31]

# Mean Absolute Deviation
mean_mad = np.mean(np.abs(data - np.mean(data)))

# Median Absolute Deviation (robust)
median_mad = median_abs_deviation(data)
```

EXAMPLE

Data: [2, 4, 6, 8, 10] Mean = 6

Absolute deviations: $|2-6|=4$, $|4-6|=2$, $|6-6|=0$, $|8-6|=2$, $|10-6|=4$

$$\text{MAD} = (4+2+0+2+4) / 5 = 12/5 = 2.4$$

INDUSTRY USE CASE

- Anomaly Detection: Median MAD is used in robust outlier detection algorithms (e.g., in sensor data from IoT devices).
- Forecasting Accuracy: MAD is used to evaluate forecast errors in supply chain and inventory management.

iv) Coefficient of Variation (CV)

DEFINITION

CV expresses standard deviation as a percentage of the mean, enabling comparison of variability across datasets with different units or scales.

$$CV = (\text{Standard Deviation} / \text{Mean}) \times 100\%$$

WHY IMPORTANT

Standard deviation alone can't compare variability between a dataset of salaries (in thousands) and one of heights (in cm). CV is unit-free — it normalizes spread relative to the mean, making cross-dataset comparisons fair.

REAL-WORLD ANALOGY

Company A: Mean salary ₹50,000, SD = ₹5,000 → CV = 10%.

Company B: Mean salary ₹5,00,000, SD = ₹40,000 → CV = 8%.

Company B is actually more consistent in pay, despite higher absolute SD!

SYNTAX (Python)

```
import numpy as np
from scipy.stats import variation

data_a = [45000, 50000, 55000, 48000, 52000]
data_b = [460000, 500000, 540000, 480000, 520000]

cv_a = variation(data_a) * 100 # CV in %
cv_b = variation(data_b) * 100
```

INDUSTRY USE CASE

- Investment: CV compares risk-to-return ratio of different assets — a lower CV means a better risk-adjusted return.
- Agriculture: CV of crop yield across fields identifies which varieties are most consistent.
- Lab Testing: CV < 5% is typically acceptable for analytical instrument precision.

1.3 Shape & Position

Shape measures describe the form of a distribution — whether it's symmetric, skewed, flat, or peaked. Position measures locate individual data points within a distribution relative to the rest.

i) Skewness — Pearson & Fisher

DEFINITION

Skewness measures the asymmetry of a distribution.

- Positive Skew (Right-skewed): Long tail to the right; mean > median > mode.
- Negative Skew (Left-skewed): Long tail to the left; mean < median < mode.
- Zero Skew: Perfectly symmetric (e.g., normal distribution).

Pearson's 1st Skewness Coefficient: $(\text{Mean} - \text{Mode}) / \text{SD}$

Pearson's 2nd Skewness Coefficient: $3 \times (\text{Mean} - \text{Median}) / \text{SD}$

Fisher's Skewness (moment-based): μ_3 / σ^3 (standardised 3rd central moment)

SUB-TOPICS

- Pearson's Skewness: Quick approximation; useful when mode is hard to define precisely.
- Fisher's Skewness (g1): Standard in most software (SciPy, Excel). More mathematically rigorous.
- Interpretation Rule: |skewness| < 0.5 = roughly symmetric; 0.5–1 = moderately skewed; > 1 = highly skewed.

WHY IMPORTANT

Skewness tells analysts whether the mean is a fair representation of center. Many statistical tests assume a normal (symmetric) distribution — knowing skewness helps decide whether to transform data or use non-parametric methods.

REAL-WORLD ANALOGY

Income distribution is positively skewed: most people earn moderate incomes, but a small number of billionaires pull the mean much higher than the median. That long right tail = positive skew.

SYNTAX (Python)

```
from scipy.stats import skew
import numpy as np

data = [1, 2, 2, 3, 3, 3, 4, 4, 10] # positive skew due to 10

fisher_skew = skew(data) # Fisher's g1
pearson_skew = 3 * (np.mean(data) - np.median(data)) / np.std(data)
```

```
# Excel: =SKEW(A1:A9) → Fisher's skewness
```

EXAMPLE

Right-skewed income data: [20k, 25k, 30k, 35k, 200k]

Mean = 62k, Median = 30k

Pearson 2nd = $3 \times (62k - 30k) / SD \rightarrow$ large positive value

Conclusion: Data is heavily right-skewed — mean is NOT representative.

INDUSTRY USE CASE

- Insurance: Claim amounts are right-skewed — most claims are small but a few catastrophic claims dominate.
- Social Media: Post engagement is right-skewed — most posts get few likes; viral posts get millions.
- Risk Management: Skewness in asset returns determines whether standard deviation understates downside risk.

ii) Kurtosis & Excess Kurtosis

DEFINITION

Kurtosis measures the 'tailedness' or 'peakedness' of a distribution — how much data is concentrated in the tails vs. the center.

$$\text{Kurtosis (Fisher / Excess)} = \mu_4 / \sigma^4 - 3$$

- Mesokurtic (Excess = 0): Normal distribution — baseline.
- Leptokurtic (Excess > 0): Heavy tails, sharp peak (e.g., t-distribution).
- Platykurtic (Excess < 0): Light tails, flat peak (e.g., uniform distribution).

Excess Kurtosis subtracts 3 (the normal distribution's kurtosis) to make the normal distribution the reference point (excess = 0).

WHY IMPORTANT

High kurtosis means extreme values (outliers) are more likely than a normal distribution would predict. This is critical in finance (fat-tailed returns), quality control, and any domain where tail events matter.

REAL-WORLD ANALOGY

A platykurtic distribution is like a low, spread hill — values are evenly spread.

A leptokurtic distribution is like a sharp mountain with heavy snow at the base — values cluster in the center AND in the extremes, with little in between.

SYNTAX (Python)

```
from scipy.stats import kurtosis
import numpy as np

data = [2, 3, 3, 4, 4, 4, 5, 5, 6, 20] # outlier at 20

excess_kurtosis = kurtosis(data, fisher=True) # Excess kurtosis (default)
full_kurtosis = kurtosis(data, fisher=False) # Full kurtosis (normal = 3)

# Excel: =KURT(A1:A10) → Excess kurtosis
```

INDUSTRY USE CASE

- Finance: Stock returns often show excess kurtosis ('fat tails'), meaning market crashes happen more often than a normal model predicts.
- Quality Engineering: Very high kurtosis in a manufacturing process signals occasional extreme defects.

iii) Percentiles, Quartiles & Deciles

DEFINITION

These are positional measures — they divide sorted data into equal groups and locate any value relative to the entire distribution.

Percentile (P_k): The value below which $k\%$ of data falls.

Quartiles: Split data into 4 equal parts — Q1 (25th), Q2 (50th=Median), Q3 (75th).

Deciles: Split data into 10 equal parts — D1 (10th) through D9 (90th).

Quintiles: Split into 5 equal parts (every 20th percentile).

WHY IMPORTANT

Percentiles let you understand where a value sits relative to all others. They are central to box plots, growth charts, salary benchmarks, test scoring, and SLA (Service Level Agreement) monitoring.

REAL-WORLD ANALOGY

If a child's height is at the 80th percentile, it means 80% of children of the same age are shorter. The child doesn't need to know the average height — the percentile tells the whole story.

SYNTAX (Python)

```
import numpy as np

data = [12, 15, 18, 20, 22, 25, 28, 30, 35, 40]

# Percentiles
p25 = np.percentile(data, 25) # Q1
p50 = np.percentile(data, 50) # Median
p75 = np.percentile(data, 75) # Q3
p90 = np.percentile(data, 90) # 90th percentile

# Quartiles together
q1, q2, q3 = np.percentile(data, [25, 50, 75])

# Excel: =PERCENTILE(A1:A10, 0.25) → Q1
```

EXAMPLE

Test Scores: [45, 55, 60, 65, 70, 75, 80, 85, 90, 95]

Q1 (25th pct) = 61.25 — bottom quarter scores below this

Q2 (50th pct) = 72.50 — median score

Q3 (75th pct) = 83.75 — top quarter scores above this

P90 = 91.25 — only top 10% score above this

INDUSTRY USE CASE

- Healthcare: Growth charts use percentiles to track child development (height, weight, BMI) relative to peers.
- Tech / SRE: P99 (99th percentile) response time is used as the 'worst case' latency SLA benchmark.
- Education: Standardised tests (CAT, GRE) report scores as percentile ranks.
- E-commerce: Revenue decile analysis segments customers from lowest (D1) to highest (D10) spenders.

iv) Z-scores & Standardisation

DEFINITION

A Z-score (standard score) measures how many standard deviations a value is away from the mean.

$$z = (x - \mu) / \sigma$$

Standardisation transforms a dataset to have mean = 0 and SD = 1. This process (also called Z-score normalisation) is fundamental for comparing variables on different scales and for ML model inputs.

SUB-TOPICS

- Z-score Interpretation: $z = 0 \rightarrow$ at the mean; $z = 2 \rightarrow$ 2 SDs above mean; $z = -1.5 \rightarrow$ 1.5 SDs below mean.
- Min-Max Normalisation (alternative): Scales values to $[0,1] \rightarrow x_{\text{scaled}} = (x - \min)/(\max - \min)$.
- Robust Scaling (alternative): Uses median and IQR instead of mean and SD — better for skewed data.
- 3-sigma Rule: In a normal distribution, ~99.7% of values fall within ± 3 standard deviations.

WHY IMPORTANT

Z-scores allow apples-to-oranges comparisons (e.g., comparing a student's Maths score with their English score). In machine learning, many algorithms (KNN, SVM, neural networks) require features to be on the same scale — standardisation ensures no single feature dominates due to magnitude alone.

REAL-WORLD ANALOGY

Imagine comparing your height to the world average. A z-score of +2 means you are 2 standard deviations taller than the average — you're in roughly the top 2.5% of heights globally.

SYNTAX (Python)

```
import numpy as np
from scipy.stats import zscore
from sklearn.preprocessing import StandardScaler

data = [50, 60, 70, 80, 90, 100]

# Z-scores with SciPy
z = zscore(data)

# Manual Z-score
z_manual = (np.array(data) - np.mean(data)) / np.std(data)

# Sklearn StandardScaler (sample std dev, ddof=1)
scaler = StandardScaler()
z_sklearn = scaler.fit_transform(np.array(data).reshape(-1, 1))

# Excel: =(A1 - AVERAGE($A$1:$A$6)) / STDEV($A$1:$A$6)
```

EXAMPLE

Scores: Mean = 70, SD = 10

Student A scored 85 $\rightarrow z = (85-70)/10 = +1.5$ (above average)

Student B scored 55 $\rightarrow z = (55-70)/10 = -1.5$ (below average)

Student A is 1.5 SDs above and Student B 1.5 SDs below the class mean.

INDUSTRY USE CASE

- Banking / Fraud Detection: Z-scores on transaction amounts flag unusually large transfers for review.
- Machine Learning: Feature standardisation is a standard preprocessing step in regression, SVM, and neural networks.
- HR Analytics: Z-scores normalize performance ratings across departments with different rating distributions.

1.4 Visualisation

Visualisation transforms numbers into pictures — allowing patterns, outliers, distributions, and relationships to be grasped instantly. A good chart communicates what tables of numbers cannot.

i) Histograms & Bin Width Selection

DEFINITION

A histogram groups continuous data into bins (intervals) and displays the frequency (count) or density of data in each bin as a bar. Unlike a bar chart, bars in a histogram are adjacent — they represent a continuous range.

Bin Width determines how wide each interval is. Too few bins → important structure is hidden. Too many bins → noise is amplified.

SUB-TOPICS — Bin Width Selection Rules

- Sturges' Rule: $k = \lceil 1 + \log_2(n) \rceil$ — simple; good for $n < 200$.
- Square Root Rule: $k = \sqrt{n}$ — quick default used in Excel.
- Freedman-Diaconis Rule: bin width = $2 \times \text{IQR} / n^{1/3}$ — robust; best for skewed or heavy-tailed data.
- Scott's Rule: bin width = $3.49 \times \sigma / n^{1/3}$ — assumes normality.

WHY IMPORTANT

Histograms are the primary tool for understanding data distribution — revealing whether it's normal, skewed, bimodal, or uniform. Bin width selection is an art: it directly affects the insight you extract.

REAL-WORLD ANALOGY

Think of grouping students by height into 5 cm bins. Bins too wide (20 cm) — you can't distinguish short vs. medium students. Bins too narrow (0.1 cm) — every student is in their own bar and the pattern is lost.

SYNTAX (Python)

```
import matplotlib.pyplot as plt
import numpy as np

data = np.random.normal(loc=170, scale=10, size=200) # heights in cm

# Auto bin selection (Freedman-Diaconis)
plt.hist(data, bins='fd', edgecolor='black', color='steelblue')
plt.title('Height Distribution'); plt.xlabel('Height (cm)');
plt.ylabel('Frequency')
plt.show()

# Manually specify bins
plt.hist(data, bins=20, density=True) # density=True → probability density
```

INDUSTRY USE CASE

- Retail: Histogram of purchase values helps decide optimal discount thresholds and pricing tiers.
- Healthcare: Histogram of patient wait times reveals whether service is normally distributed or skewed.
- Finance: Return distributions help assess whether a portfolio's performance follows expected patterns.

ii) Box Plots & Violin Plots

DEFINITION

Box Plot (Box-and-Whisker Plot): A standardised way to display the distribution of a dataset using five summary statistics.

Components: Minimum | Q1 | Median | Q3 | Maximum (plus outlier dots)

Whiskers extend to $Q1 - 1.5 \times IQR$ and $Q3 + 1.5 \times IQR$.

Violin Plot: Combines a box plot with a rotated kernel density estimate (KDE) on each side, showing the full shape of the distribution. Wider sections indicate more data at that value.

SUB-TOPICS

- Five-Number Summary: Min, Q1, Median, Q3, Max — the core of any box plot.
- Outlier Dots: Points beyond the whiskers are plotted individually.
- Side-by-Side Box Plots: Perfect for comparing distributions across multiple groups or categories.
- Notched Box Plot: Notch around the median indicates 95% confidence interval for the median.

WHY IMPORTANT

Box plots pack the five-number summary into a single compact graphic — ideal for comparing multiple groups side by side. Violin plots add distribution shape information, revealing bimodality and density that box plots hide.

REAL-WORLD ANALOGY

A box plot is like a temperature report: min, max, and the middle 50% range. A violin plot is like a detailed weather forecast that also shows how likely each temperature is throughout the day.

SYNTAX (Python)

```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import numpy as np

np.random.seed(42)
df = pd.DataFrame({'Group A': np.random.normal(60, 10, 100),
                  'Group B': np.random.normal(75, 15, 100)})

# Box Plot (Matplotlib)
df.boxplot(); plt.title('Box Plot'); plt.show()

# Violin Plot (Seaborn)
df_long = df.melt(var_name='Group', value_name='Score')
sns.violinplot(x='Group', y='Score', data=df_long, inner='quartile')
plt.title('Violin Plot'); plt.show()
```

INDUSTRY USE CASE

- A/B Testing: Box plots compare control vs. treatment group metric distributions to evaluate experiment results.
- HR Analytics: Violin plots reveal whether salary distributions within departments are unimodal or bimodal.
- Sports Analytics: Box plots compare player performance metrics across teams or seasons.

iii) Stem-and-Leaf Plots & QQ Plots

DEFINITION

Stem-and-Leaf Plot: A text-based histogram where each data value is split into a 'stem' (leading digit(s)) and a 'leaf' (trailing digit). Retains every individual data value while showing distribution shape.

QQ Plot (Quantile-Quantile Plot): A scatter plot that compares the quantiles of your data

against the quantiles of a theoretical distribution (usually normal). If data follows that distribution, points fall on a straight diagonal line.

SUB-TOPICS

- Stem-and-Leaf: Best for small datasets ($n < 50$). Data remains readable and sortable.
- Back-to-Back Stem-and-Leaf: Two groups share stems; leaves branch left and right — ideal for comparison.
- Normal QQ Plot: Points on the diagonal → data is normally distributed. Curved → skewed. S-shaped → heavy tails.
- QQ Plot vs. PP Plot: QQ focuses on tails; PP focuses on the middle of the distribution.

WHY IMPORTANT

Stem-and-leaf plots preserve raw data in a visual format — great for small exploratory datasets in class or quick checks. QQ plots are indispensable for testing the normality assumption required by t-tests, ANOVA, and linear regression.

REAL-WORLD ANALOGY

A QQ plot is like comparing your city's temperature distribution against the 'perfect normal curve.' If your city's winters are colder and summers hotter than normal (heavy tails), the QQ plot shows an S-curve, not a straight line.

SYNTAX (Python)

```
import numpy as np
import scipy.stats as stats
import matplotlib.pyplot as plt

data = np.random.normal(loc=50, scale=10, size=100)

# Normal QQ Plot
stats.probplot(data, dist='norm', plot=plt)
plt.title('Normal QQ Plot'); plt.show()

# Stem-and-leaf (text-based, for small data)
# Use the 'stemgraphic' library or print manually
scores = [72, 75, 78, 81, 83, 85, 88, 90, 92]
# Stem 7: 2 5 8 | Stem 8: 1 3 5 8 | Stem 9: 0 2
```

INDUSTRY USE CASE

- Academic Research: QQ plots validate the normality assumption before applying parametric statistical tests.
- Finance: QQ plots of stock returns reveal fat tails — indicating crash risk is underestimated by normal models.
- Clinical Trials: QQ plots check if residuals from regression models are normally distributed.

iv) Frequency Tables & Contingency Tables

DEFINITION

Frequency Table: A tabular summary counting how often each value or category appears.

Includes:

- Frequency (count)
- Relative Frequency (proportion = count / total)
- Cumulative Frequency (running total)

Contingency Table (Cross-tabulation / Crosstab): A 2D frequency table showing the joint distribution of two categorical variables. Cells show counts or proportions for each combination of categories.

SUB-TOPICS

- Grouped Frequency Table: Used for continuous variables — groups values into class intervals.
- Marginal Frequencies: Row or column totals in a contingency table.
- Expected Frequencies: Theoretical counts under the null hypothesis — used in Chi-Square tests.
- Chi-Square Test of Independence: Tests whether two categorical variables in a contingency table are related.

WHY IMPORTANT

Frequency tables are the foundation for categorical data analysis. Contingency tables reveal relationships between categorical variables and are the starting point for Chi-Square tests, which determine statistical association — e.g., does gender influence product preference?

REAL-WORLD ANALOGY

A restaurant's menu sales log can be turned into a frequency table (how often each dish is ordered). A contingency table might cross-tabulate dish type vs. time of day — revealing that pasta dominates at lunch but steak dominates at dinner.

SYNTAX (Python)

```
import pandas as pd
from scipy.stats import chi2_contingency

df = pd.DataFrame({
    'Gender': ['M', 'F', 'M', 'F', 'M', 'F', 'M', 'F'],
    'Product': ['A', 'B', 'A', 'A', 'B', 'B', 'A', 'B']
})
```

```
# Frequency Table
freq_table = df['Product'].value_counts().reset_index()
freq_table.columns = ['Product', 'Frequency']
freq_table['Relative_Freq'] = freq_table['Frequency'] / len(df)

# Contingency Table
contingency = pd.crosstab(df['Gender'], df['Product'], margins=True)
print(contingency)

# Chi-Square Test of Independence
chi2, p, dof, expected = chi2_contingency(contingency.iloc[:-1, :-1])
print(f"Chi2={chi2:.3f}, p-value={p:.3f}")
```

EXAMPLE

Contingency Table — Gender vs. Product Preference:

Product A	Product B	Total
Male 3	1	4
Female 1	3	4
Total 4	4	8

Chi-Square test: $p < 0.05 \rightarrow$ Gender and Product preference are related.

INDUSTRY USE CASE

- Marketing: Contingency tables analyse whether customer segment (age group, region) influences campaign response rates.
- Healthcare: Cross-tabulate treatment type vs. recovery outcome to assess treatment effectiveness.
- E-commerce: Frequency tables of page visit counts guide UX improvements and content prioritization.
- Politics / Surveys: Contingency tables in surveys cross-tabulate voting preference by demographic groups.

Quick Reference — Unit 1 Summary

Use this table as a cheat sheet for choosing the right measure or chart:

Concept	Best Used When	Avoid When
Mean	Data is symmetric, no outliers	Heavy outliers present
Median	Skewed data or outliers present	Need further arithmetic
Mode	Categorical data; finding commonest	Continuous data (rare repeats)
Trimmed Mean	Outliers exist but you need a mean	Small datasets (too much trimmed)

IQR	Robust spread; outlier fences needed	When full range matters
Std Deviation	Normal or near-normal distributions	Highly skewed data
CV	Comparing spread across diff. scales	Mean ≈ 0 (CV $\rightarrow$ infinity)
Z-score	Standardising, detecting outliers	Non-normal distributions
Histogram	Continuous data distribution	Categorical data
Box Plot	Comparing multiple groups' spread	Showing exact distribution shape
Violin Plot	Full distribution shape + groups	Non-technical audiences
QQ Plot	Testing normality assumption	Casual exploratory analysis
Contingency Table	Relationship between 2 cat. variables	Continuous variables directly

UNIT 1 — Descriptive Statistics

Interview-Base Unit Test

LEVEL 1 — WARMUP TEST | 5 Multiple Choice Questions

Instructions: Choose the best answer for each question. The correct option is marked with .

Q1

Which measure of central tendency is MOST resistant to extreme outliers in a dataset?

- A) Mean
- B) Median
- C) Mode
- D) Trimmed Mean



Answer:

Q2

What does IQR (Interquartile Range) measure, and how is it calculated?

- A) The average of all values: (Sum / n)
- B) The range of the full dataset: Max – Min
- C) The spread of the middle 50% of data: Q3 – Q1
- D) The standard deviation divided by the mean



Answer:

Q3

In a positively skewed (right-skewed) distribution, which of the following correctly describes the relationship between mean, median, and mode?

- A) Mean < Median < Mode
- B) Mode < Median < Mean
- C) Mean = Median = Mode
- D) Median < Mode < Mean



Answer:

Q4

Why does sample variance use (n – 1) in the denominator instead of n? This correction is known as:

- A) Pearson's correction — to normalize the variance
- B) Bessel's correction — to remove bias in estimating population variance
- C) Fisher's correction — to account for skewness
- D) Scott's correction — to adjust for bin width



Answer:

Q5

Which bin-width selection rule for histograms is considered MOST robust for skewed or heavy-tailed data?

- A) Sturges' Rule: $k = 1 + \log_2(n)$
- B) Square Root Rule: $k = \sqrt{n}$

C) Scott's Rule: bin width = $3.49 \times \sigma / n^{1/3}$

D) Freedman-Diaconis Rule: bin width = $2 \times \text{IQR} / n^{1/3}$



Answer:



LEVEL 2 — INTERVIEWER TEST | 15 Intermediate to Semi-Advanced Questions

Format: Each question includes a conversational student-style answer with code snippets and real-world reasoning.



Section A — Descriptive Questions (4 Questions)

Q1

Explain the difference between Population Variance (σ^2) and Sample Variance (s^2). Why is Bessel's correction necessary, and when would you use each in a data analysis project?

 **Answer**

Learnlytics
— handbook

LEARN SMART. ANALYZE BETTER. GROW FASTER.

Q2

Explain Skewness in statistical distributions. Describe the types of skewness, the difference between Pearson's and Fisher's skewness coefficients, and why understanding skewness is critical before applying statistical tests.

✓ **Answer**



Q3

Explain Z-scores and standardisation. What does a Z-score represent, what are its key applications in machine learning and data analytics, and how does it differ from Min-Max normalisation?

✓ **Answer**

Learnlvtics

LEARN SMART. ANALYZE BETTER. GROW FASTER.

Q4

Compare and contrast Kurtosis and Skewness as shape measures. What does excess kurtosis reveal that skewness does not, and why is high kurtosis particularly dangerous in financial risk modelling?

☒ Answer



Learnlytics
— .handbook

LEARN SMART. ANALYZE BETTER. GROW FASTER.



Section B — Scenario-Based Descriptive Questions (4 Questions)

Q5

SCENARIO: You are an HR analyst at a tech company with 50 employees. 48 employees earn between ₹40,000–₹80,000/month, but the CEO earns ₹30,00,000/month and the CTO earns ₹20,00,000/month. A reporter asks you: 'What is the average salary at your company?' — How would you respond, and which measure(s) would you use and why?

✓ Answer



Q6

SCENARIO: You are building a machine learning model to predict house prices. Your dataset has three features: House Size (200–5,000 sq ft), Number of Bedrooms (1–10), and Annual Household Income (₹2,00,000–₹2,00,00,000). What preprocessing steps would you apply, and why do they matter for ML model performance?

✓ Answer

Q7

SCENARIO: You are a Site Reliability Engineer (SRE) monitoring API response times. Typical responses take 50–150ms, but occasionally a request spikes to 8,000ms or more. Your manager asks: 'What is our typical response time, and are we meeting our SLA of under 200ms for 95% of requests?' — Walk through your analysis approach.

☒ Answer



Learnlytics
— .handbook

LEARN SMART. ANALYZE BETTER. GROW FASTER.

Q8

SCENARIO: A data analyst at a survey research firm notices that responses to a satisfaction scale (1–10) contain many '1' and '10' responses that appear to be troll or

random entries, distorting the overall mean. Your task is to produce a robust central estimate. Which statistical technique would you apply, and how does it compare to simply removing outliers?

☒ Answer



Learnlvtics

 Section C — Coding Questions (4 Questions)

.handbook

Q9

Write a Python function that accepts a list of numbers and returns a comprehensive summary of all Measures of Center: Mean, Median, Mode, Trimmed Mean (10%), Winsorized Mean (10%), Midrange, and Mid-Quartile. Include proper handling for edge cases.

☒ Answer

Q10

Write a Python function to perform full outlier detection on a dataset using both the IQR Fence method and Z-score method (threshold = 3). The function should return flagged outliers from each method, the count, and whether the two methods agree.

☒ Answer



Learnlytics
.handbook

LEARN SMART. ANALYZE BETTER. GROW FASTER.

Q11

Write Python code to generate a side-by-side Box Plot and Violin Plot comparing the score distributions of three student groups (Group A, B, and C). Add proper labels,

colour styling, and annotate the median line for each group.

☒ Answer



Q12

Write Python code to generate and visualise a QQ Plot for a dataset of exam scores. Then demonstrate what happens to the QQ plot when the same data is log-transformed — explain the interpretation at each step.

☒ Answer

Learn Analytics
handbook
LEARN SMART. ANALYZE BETTER. GROW FASTER.

 Section D — Database Scenario-Based Coding Questions (3 Questions)

Q13

DATABASE SCENARIO: You have a table called `employee_salaries` with columns: `employee_id`, `department`, `salary`, and `hire_year`. Write SQL queries to compute the Mean, Median (approximate), Mode, and Standard Deviation of salaries — both overall and grouped by department.

☒ **Answer**



Q14

DATABASE SCENARIO: You have a `customer_orders` table with columns: `order_id`, `customer_id`, `product_category`, `customer_segment` ('Premium', 'Standard', 'Basic'),

and order_total. Write SQL to build a Contingency Table of customer_segment vs. product_category, include row/column totals, and identify which segment-category combination has the highest purchase frequency.

☒ Answer



Q15

DATABASE SCENARIO: You have a sensor_readings table with columns: sensor_id, timestamp, and temperature_celsius. Write SQL to compute the IQR for each sensor, identify outlier readings using the $1.5 \times \text{IQR}$ fence rule, and return only the outlier rows with their deviation from the median.

☒ Answer



Learnlytics

.handbook

LEARN SMART. ANALYZE BETTER. GROW FASTER.

UNIT 1 — Descriptive Statistics

Interview-Base Unit Test

LEVEL 1 — WARMUP TEST | 5 Multiple Choice Questions

Instructions: Choose the best answer for each question. The correct option is marked with ✓.

Q1	Which measure of central tendency is MOST resistant to extreme outliers in a dataset?
	A) Mean B) Median ✓ C) Mode D) Trimmed Mean
	Explanation: The Median is the most resistant to outliers because it only depends on the middle value(s) of sorted data — extreme values cannot shift it significantly. The Mean is pulled toward outliers. The Trimmed Mean also resists outliers but requires removing data points.
Q2	What does IQR (Interquartile Range) measure, and how is it calculated?
	A) The average of all values: (Sum / n) B) The range of the full dataset: $\text{Max} - \text{Min}$ C) The spread of the middle 50% of data: $Q3 - Q1$ ✓ D) The standard deviation divided by the mean
	Explanation: $\text{IQR} = Q3 - Q1$, representing the range of the middle 50% of data. It is robust against outliers because it ignores the bottom 25% and top 25% of values. It is also used in the whisker fences of box plots to flag outliers: below $Q1 - 1.5 \times \text{IQR}$ or above $Q3 + 1.5 \times \text{IQR}$.

Q3	In a positively skewed (right-skewed) distribution, which of the following correctly describes the relationship between mean, median, and mode?
	A) Mean < Median < Mode B) Mode < Median < Mean ✓ C) Mean = Median = Mode D) Median < Mode < Mean
	Explanation: In a right-skewed distribution, the long tail pulls the Mean to the right, making it the largest. The Median sits in the middle, and the Mode (peak of distribution) is the smallest. So: Mode < Median < Mean. Income distributions are a classic example of this pattern.

Q4	Why does sample variance use $(n - 1)$ in the denominator instead of n ? This correction is known as:
	A) Pearson's correction — to normalize the variance B) Bessel's correction — to remove bias in estimating population variance ✓ C) Fisher's correction — to account for skewness D) Scott's correction — to adjust for bin width
	Explanation: Bessel's correction (dividing by $n - 1$ instead of n) removes bias when estimating the population variance from a sample. A sample tends to underestimate the true spread because it does not fully capture the extremes of the population. Subtracting 1 degree of freedom corrects this bias.

LEARN SMART. ANALYZE BETTER. GROW FASTER.

Q5	Which bin-width selection rule for histograms is considered MOST robust for skewed or heavy-tailed data?
	A) Sturges' Rule: $k = 1 + \log_2(n)$ B) Square Root Rule: $k = \sqrt{n}$ C) Scott's Rule: bin width = $3.49 \times \sigma / n^{(1/3)}$ D) Freedman-Diaconis Rule: bin width = $2 \times \text{IQR} / n^{(1/3)}$ ✓



Explanation: The Freedman-Diaconis Rule uses the IQR instead of standard deviation, making it robust against outliers and skewness. Scott's Rule assumes normality and breaks down with skewed data. Sturges' Rule is only suitable for small, roughly normal datasets ($n < 200$).

LEVEL 2 — INTERVIEWER TEST | 15 Intermediate to Semi-Advanced Questions

Format: Each question includes a conversational student-style answer with code snippets and real-world reasoning.

Section A — Descriptive Questions (4 Questions)

Q1

Explain the difference between Population Variance (σ^2) and Sample Variance (s^2). Why is Bessel's correction necessary, and when would you use each in a data analysis project?

Answer

Great question — this trips up a lot of people! The core difference is about what we're measuring and whether we have the full picture.

Population Variance (σ^2) is used when you have data for every single member of the group you're interested in. You divide by N (the full count). Sample Variance (s^2) is used when you only have a subset of the population, which is almost always the case in real data work. You divide by $(n - 1)$ instead of n .

Why $n - 1$? Here's the intuition: a sample tends to cluster closer to its own mean than the full population does. So if you just divide by n , you'd consistently underestimate the true spread. Subtracting 1 gives back that 'lost' degree of freedom and removes the bias. This is called Bessel's

correction.

```
import numpy as np
```

```
data = [4, 7, 13, 2, 1]
```

```
# Population variance (you surveyed EVERY employee in a small team)
```

```
pop_var = np.var(data, ddof=0) # ddof=0 → divide by N
```

```
# Sample variance (you surveyed 5 out of 500 employees)
```

```
samp_var = np.var(data, ddof=1) # ddof=1 → divide by N-1 (Bessel's correction)
```

```
print(f'Population Variance: {pop_var:.2f}')
```

```
print(f'Sample Variance: {samp_var:.2f}')
```

Real-world rule of thumb: In almost every data science project you use sample variance (ddof=1) because you're working with collected data, not a census. Population variance is rare — maybe a full database of all transactions in one month, or a fixed controlled experiment where you have all subjects.

handbook

LEARN SMART. ANALYZE BETTER. GROW FASTER.

Q2

Explain Skewness in statistical distributions. Describe the types of skewness, the difference between Pearson's and Fisher's skewness coefficients, and why understanding skewness is critical before applying statistical tests.

✓ Answer

Skewness is one of those concepts that sounds simple but has real implications for how you analyse

data. It measures the asymmetry of a distribution around its mean.

There are three types: Positive (right) skew — the tail stretches to the right, $\text{mean} > \text{median} > \text{mode}$. Think income data: most people earn moderate amounts, but a few billionaires pull the mean way up. Negative (left) skew — the tail drags left, $\text{mean} < \text{median} < \text{mode}$. Think retirement age: most people retire around 60–65, but a few retire very early. Zero skew — perfectly symmetric, like an ideal normal distribution.

Now, Pearson vs Fisher: Pearson's coefficient is a shortcut approximation. Pearson's 1st: $(\text{Mean} - \text{Mode}) / \text{SD}$. Pearson's 2nd: $3 \times (\text{Mean} - \text{Median}) / \text{SD}$ — useful when mode is hard to define. Fisher's skewness (g_1) is the mathematically proper version based on the 3rd standardised central moment. It's what scipy, Excel's `=SKEW()`, and most software actually compute.

```
from scipy.stats import skew

import numpy as np

# Positively skewed income data (values in thousands)
data = [20, 25, 30, 35, 200]

fisher_skew = skew(data) # Fisher's g1

pearson_skew = 3 * (np.mean(data) - np.median(data)) / np.std(data) #
Pearson's 2nd

print(f'Fisher Skewness: {fisher_skew:.3f}') # > 1 → highly right-skewed
print(f'Pearson Skewness: {pearson_skew:.3f}')
```

Interpretation rule: $|\text{skewness}| < 0.5$ = roughly symmetric; $0.5\text{--}1.0$ = moderate skew; > 1.0 = high skew. Why it matters: many tests like t-tests, ANOVA, and linear regression assume roughly normal (symmetric) distributions. If your data is heavily skewed, results may be misleading — you'd want to

log-transform the data or use non-parametric alternatives.

Q3

Explain Z-scores and standardisation. What does a Z-score represent, what are its key applications in machine learning and data analytics, and how does it differ from Min-Max normalisation?

 Answer

Z-scores are a powerful, versatile tool — I think of them as translating every data point into a universal language of 'how unusual is this?'

A Z-score tells you how many standard deviations a value is from the mean: $z = (x - \mu) / \sigma$. If $z = 0$, you're exactly average. $z = +2$ means you're 2 standard deviations above average. $z = -1.5$ means 1.5 standard deviations below average. In a normal distribution, about 99.7% of values fall within ± 3 standard deviations — the famous 3-sigma rule.

Standardisation (Z-score normalisation) transforms an entire dataset to have mean = 0 and SD = 1. This is critical in ML because algorithms like KNN, SVM, and neural networks are distance-based or gradient-based — features on vastly different scales (salary in lakhs vs. age in years) would cause the salary feature to dominate unfairly.

```
import numpy as np

from sklearn.preprocessing import StandardScaler

data = np.array([50, 60, 70, 80, 90, 100]).reshape(-1, 1)

# Manual Z-score (uses population std by default)
z_manual = (data - np.mean(data)) / np.std(data)
```

```
# Sklearn StandardScaler (uses sample std, ddof=1)
```

```
scaler = StandardScaler()
```

```
z_sklearn = scaler.fit_transform(data)
```

```
print('Z-scores:', z_manual.flatten().round(2))
```

Z-score vs. Min-Max normalisation: Z-scores preserve the shape of the distribution and handle outliers reasonably. Min-Max ($x_{\text{scaled}} = (x - \min) / (\max - \min)$) squishes everything to $[0, 1]$ but is very sensitive to outliers — one extreme value compresses all others. Use Z-scores for most ML pipelines; use Min-Max when you know the range is bounded and meaningful (like pixel intensities 0–255).

Real use cases: Fraud detection (flag transactions with $z > 3$), HR analytics (compare performance scores across departments), feature preprocessing for regression, SVM, and neural networks.

Q4

Compare and contrast Kurtosis and Skewness as shape measures. What does excess kurtosis reveal that skewness does not, and why is high kurtosis particularly dangerous in financial risk modelling?

☒ **Answer**

Skewness and kurtosis are both shape descriptors, but they capture completely different things — skewness is about asymmetry, kurtosis is about tail heaviness.

Skewness tells you: is the distribution leaning left or right? Is the bulk of the data concentrated on one side? Kurtosis asks a different question: are extreme values (outliers) more or less common than in a normal distribution?

Excess kurtosis = $\mu_4/\sigma^4 - 3$. The '-3' makes the normal distribution the reference point (excess = 0).
Leptokurtic (excess > 0): sharp peak, heavy tails — extreme events happen more often than normal.
Platykurtic (excess < 0): flat peak, light tails — values are spread more evenly.

```
from scipy.stats import kurtosis
```

```
import numpy as np
```

```
# Normal-ish data
```

```
normal_data = np.random.normal(0, 1, 1000)
```

```
# Fat-tailed data (adding outliers)
```

```
fat_tail_data = np.concatenate([np.random.normal(0, 1, 990), [10, -10, 15, -12, 20, -18, 8, -9, 11, -14]])
```

```
print(f'Normal-ish Excess Kurtosis: {kurtosis(normal_data, fisher=True):.3f}')  
# ≈ 0
```

```
print(f'Fat-tail Excess Kurtosis: {kurtosis(fat_tail_data, fisher=True):.3f}')  
# > 0
```

Why it's dangerous in finance: Standard risk models (like Value at Risk using normal distribution) assume kurtosis ≈ 0 . But stock returns consistently show positive excess kurtosis — 'fat tails'. This means market crashes and extreme rallies happen far more often than the normal model predicts. The 2008 financial crisis was partly fuelled by models that ignored fat tails and underestimated tail risk. High kurtosis = the 'black swan' problem — rare but catastrophic events.



Section B — Scenario-Based Descriptive Questions (4 Questions)

Q5

SCENARIO: You are an HR analyst at a tech company with 50 employees. 48 employees earn between ₹40,000–₹80,000/month, but the CEO earns ₹30,00,000/month and the CTO earns ₹20,00,000/month. A reporter asks you: 'What is the average salary at your company?' — How would you respond, and which measure(s) would you use and why?

✓ Answer

This is a classic case where the word 'average' is ambiguous and the choice of measure has ethical and practical implications.

If I just report the mean, it would be something like ₹1,38,000/month (heavily inflated by the CEO/CTO). That's technically correct but misleading — it implies the 'typical' employee earns well above what they actually do. A candidate offered a job would feel deceived.

I'd report the median salary. With 50 data points, the median would likely fall around ₹55,000–65,000/month, which actually reflects where most employees sit. I'd say: 'The median salary is ₹X — this represents what a typical employee earns. Our mean salary appears higher due to senior executive compensation.'

```
import numpy as np
```

```
# Simulated salaries
```

```
salaries = [40000 + i * 1000 for i in range(48)] + [2000000, 3000000] # 48  
staff + 2 execs
```

```
print(f'Mean Salary: ₹{np.mean(salaries):,.0f}')
```

```
print(f'Median Salary: ₹{np.median(salaries):,.0f}')
```

```
print(f'CV (spread): {(np.std(salaries)/np.mean(salaries)*100):.1f}%') # Very  
high CV = unequal distribution
```

Additionally, I might include the IQR (Q1–Q3 range) to show where the 'middle 50%' of employees fall — this gives the full picture without a single number being misleading. If the reporter insists on 'average', I'd use the trimmed mean (dropping the top 5%) as a compromise that gives a mean-like figure without the outlier distortion.

Key takeaway: The right measure depends on your audience and the story the data tells. In salary reporting, median is the industry standard precisely for this reason.

Q6

SCENARIO: You are building a machine learning model to predict house prices. Your dataset has three features: House Size (200–5,000 sq ft), Number of Bedrooms (1–10), and Annual Household Income (₹2,00,000–₹2,00,00,000). What preprocessing steps would you apply, and why do they matter for ML model performance?

✓ Answer

Feature scaling is one of those preprocessing steps that sounds optional but is actually essential for many algorithms. Here's how I'd think through this scenario.

The problem: House Size ranges 200–5000, Bedrooms 1–10, and Income 2,00,000–2,00,00,000. If I feed these raw into a KNN, SVM, or gradient descent-based model (linear regression, neural net), the income feature will completely dominate simply because its numbers are orders of magnitude larger — not because it's more important.

Step 1 — Check for skewness first. Income is likely heavily right-skewed (most households are moderate, few are very high). I'd apply a log transformation to income: $\log(\text{income})$. This brings it closer to normal before scaling.

```
import numpy as np
```

```
import pandas as pd
```

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler
```

```
# Simulate feature data
```

```
df = pd.DataFrame({
```

```
    'house_size': np.random.uniform(200, 5000, 100),
```

```
    'bedrooms': np.random.randint(1, 10, 100).astype(float),
```

```
    'income': np.random.lognormal(mean=13, sigma=1.5, size=100) # right-skewed
```

```
})
```

```
# Step 1: Log-transform skewed income
```

```
df['income_log'] = np.log(df['income'])
```

```
# Step 2: Standardise all features (Z-score) – best for SVM, KNN, Neural Nets
```

```
scaler = StandardScaler()
```

```
df_scaled =
```

```
pd.DataFrame(scaler.fit_transform(df[['house_size', 'bedrooms', 'income_log']]),
```

```
    columns=['size_z', 'beds_z', 'income_z'])
```

Step 2 — Standardise with Z-scores (preferred) or Min-Max scale. I'd use StandardScaler for most algorithms because it handles outliers better than Min-Max. The Coefficient of Variation (CV) is useful here to see which feature had the most relative spread before scaling.

Key insight: After scaling, all three features have mean ≈ 0 and SD ≈ 1 — now no feature dominates due to scale alone, and the model can learn which features actually predict price.

Q7

SCENARIO: You are a Site Reliability Engineer (SRE) monitoring API response times. Typical responses take 50–150ms, but occasionally a request spikes to 8,000ms or

more. Your manager asks: 'What is our typical response time, and are we meeting our SLA of under 200ms for 95% of requests?' — Walk through your analysis approach.

✓ Answer

This is a perfect scenario where naive use of the mean would give a completely misleading picture, and percentile-based analysis is the right tool.

First, I'd acknowledge that this data is heavily right-skewed — most responses are fast but there's a long tail of slow requests. The mean will be dragged upward by those 8,000ms spikes and won't represent typical performance. So I'd immediately look at the median (P50) and key percentiles.

For the SLA question specifically: 'Under 200ms for 95% of requests' is a P95 SLA. I need to compute the 95th percentile response time and check if it's under 200ms. In SRE practice, we care about P50 (typical), P95 (near-worst case), and P99 (true worst case).

```
import numpy as np

from scipy.stats import iqr

# Simulated response times (ms)

np.random.seed(42)

response_times = np.concatenate([

    np.random.normal(100, 25, 950), # 950 normal requests

    np.random.uniform(2000, 8000, 50) # 50 slow spikes

])

print(f'Mean (P50): {np.mean(response_times):.1f}ms ← inflated by spikes')

print(f'Median: {np.percentile(response_times, 50):.1f}ms ← true typical')

print(f'P95 (SLA check): {np.percentile(response_times, 95):.1f}ms ← SLA =
```

```
200ms')
```

```
print(f'P99: {np.percentile(response_times, 99):.1f}ms ← worst case')
```

```
# IQR-based outlier detection
```

```
q1, q3 = np.percentile(response_times, [25, 75])
```

```
iqr_val = iqr(response_times)
```

```
outliers = response_times[response_times > q3 + 1.5 * iqr_val]
```

```
print(f'Outlier count: {len(outliers)} requests flagged')
```

I'd report: 'Our median response time is ~100ms (well within SLA). Our P95 is Xms — if this is under 200ms, we are meeting SLA for 95% of requests. However, P99 spikes to Yms, suggesting infrastructure issues for a small fraction of users. I recommend alerting on P99 > 500ms to catch these before users notice.'

This percentile-based reporting is standard in production SRE and is far more actionable than mean-based reporting.

Learnlvtics

Q8

SCENARIO: A data analyst at a survey research firm notices that responses to a satisfaction scale (1–10) contain many '1' and '10' responses that appear to be troll or random entries, distorting the overall mean. Your task is to produce a robust central estimate. Which statistical technique would you apply, and how does it compare to simply removing outliers?

☒ **Answer**

This is a nuanced situation — we have data quality issues but we can't just arbitrarily delete records without justification. That would be cherry-picking.

I'd use either a Trimmed Mean or a Winsorized Mean. Let me explain the difference and trade-offs.

Trimmed Mean: remove the bottom and top k% of values, then compute the mean on the remainder.

Winsorized Mean: instead of removing extremes, replace them with the boundary value at the k-th percentile — so a '1' response gets replaced with the 5th percentile value, and a '10' gets capped at the 95th percentile.

```
import numpy as np

from scipy.stats import trim_mean, mstats

# Satisfaction scores: mostly 4-8, with troll 1s and 10s

np.random.seed(0)

scores = list(np.random.randint(4, 9, 90)) + [1]*5 + [10]*5 # 10% extreme

raw_mean = np.mean(scores)

trimmed = trim_mean(scores, proportiontocut=0.10) # Drop bottom & top 10%

winsorized_arr = mstats.winsorize(scores, limits=[0.10, 0.10])

winsorized = np.mean(winsorized_arr)

median_val = np.median(scores)

print(f'Raw Mean: {raw_mean:.2f} ← distorted')

print(f'Trimmed Mean: {trimmed:.2f} ← removes extremes')

print(f'Winsorized Mean: {winsorized:.2f} ← caps extremes')

print(f'Median: {median_val:.2f} ← most robust')
```

Why Winsorized is often better than Trimmed here: Winsorization keeps all data points in the analysis (sample size stays intact) and doesn't discard potentially valid boundary responses entirely — it just reduces their influence. In survey research, this is considered methodologically cleaner than deletion.

When to use each: Trimmed mean = when outliers are clearly erroneous (data entry errors). Winsorized mean = when extremes are real but exaggerated (troll responses, human bias). Median = when you want maximum robustness with a single number. In my report, I'd present all three and explain the choice transparently.

Section C — Coding Questions (4 Questions)

Q9

Write a Python function that accepts a list of numbers and returns a comprehensive summary of all Measures of Center: Mean, Median, Mode, Trimmed Mean (10%), Winsorized Mean (10%), Midrange, and Mid-Quartile. Include proper handling for edge cases.

Answer

Here's a clean, production-ready implementation. I've structured it as a function that returns a dictionary so each result is clearly labelled and easy to use downstream.

```
import numpy as np

import statistics as st

from scipy.stats import trim_mean, mstats


def center_of_mass_summary(data: list) -> dict:
    '''
    Returns all measures of center for a numeric dataset.
    Handles edge cases: empty list, single value, no mode.
    '''
```

```

if not data:

raise ValueError('Input data list is empty.')


arr = np.array(data, dtype=float)

n = len(arr)


# --- Basic Measures ---

mean = np.mean(arr)

median = np.median(arr)


# Mode: handle multimodal case gracefully

try:

mode = st.mode(arr)

except st.StatisticsError:

mode = None # No unique mode


# --- Robust Means ---

prop = 0.10 if n >= 10 else 0 # Only trim if enough data

trimmed_mean = trim_mean(arr, proportiontocut=prop)

winsorized_data = mstats.winsorize(arr, limits=[prop, prop])

winsorized_mean = np.mean(winsorized_data)


# --- Range-based Centers ---

midrange = (np.min(arr) + np.max(arr)) / 2

q1, q3 = np.percentile(arr, [25, 75])

```

```
mid_quartile = (q1 + q3) / 2
```

```
return {
```

```
    'n': n,
```

```
    'mean': round(mean, 4),
```

```
    'median': round(median, 4),
```

```
    'mode': mode,
```

```
    'trimmed_mean_10': round(trimmed_mean, 4),
```

```
    'winsorized_mean': round(winsorized_mean, 4),
```

```
    'midrange': round(midrange, 4),
```

```
    'mid_quartile': round(mid_quartile, 4),
```

```
}
```

```
# — Test with a right-skewed dataset —
```

```
salaries = [30000, 32000, 35000, 40000, 42000, 45000, 50000, 55000, 60000,  
            500000]
```

```
result = center_of_mass_summary(salaries)
```

```
for key, value in result.items():
```

```
    print(f'{key:25s}: {value}')
```

Key design decisions: I check for empty input early. The mode handling uses a try/except because statistics.mode raises StatisticsError on multimodal data — returning None is more graceful than crashing. I also guard the trimming proportion (only trim if $n \geq 10$, otherwise trimming removes too much data). Returning a dict makes it easy to extend — you can unpack it into a pandas DataFrame row for batch analysis.

Q10

Write a Python function to perform full outlier detection on a dataset using both the IQR Fence method and Z-score method (threshold = 3). The function should return flagged outliers from each method, the count, and whether the two methods agree.

✓ Answer

Outlier detection is something every data analyst does constantly, and it's worth having a reusable, well-documented function for it. I'll implement both methods and compare their results.

```
import numpy as np

from scipy.stats import zscore


def detect_outliers(data: list, z_threshold: float = 3.0) -> dict:
    '''
    Detects outliers using IQR Fence and Z-score methods.
    Returns flagged indices, values, and method agreement.
    '''
    arr = np.array(data, dtype=float)

    # — Method 1: IQR Fence —
    q1, q3 = np.percentile(arr, [25, 75])
    iqr_val = q3 - q1
    lower = q1 - 1.5 * iqr_val
    upper = q3 + 1.5 * iqr_val
    iqr_mask = (arr < lower) | (arr > upper)
```

```
# — Method 2: Z-score ( $|z| > \text{threshold}$ ) —
```

```
z_scores = zscore(arr)
```

```
z_mask = np.abs(z_scores) > z_threshold
```

```
# — Agreement Analysis —
```

```
agreed = iqr_mask & z_mask # Both flag it
```

```
iqr_only = iqr_mask & ~z_mask # Only IQR flags it
```

```
z_only = ~iqr_mask & z_mask # Only Z-score flags it
```

```
return {
```

```
'iqr_fences': (round(lower, 2), round(upper, 2)),
```

```
'iqr_outliers': arr[iqr_mask].tolist(),
```

```
'iqr_count': int(iqr_mask.sum()),
```

```
'z_outliers': arr[z_mask].tolist(),
```

```
'z_count': int(z_mask.sum()),
```

```
'agreed_outliers': arr[agreed].tolist(),
```

```
'iqr_only': arr[iqr_only].tolist(),
```

```
'z_only': arr[z_only].tolist(),
```

```
}
```

```
# — Test —
```

```
test_data = [10, 12, 11, 13, 14, 12, 11, 10, 200, -50, 15, 13, 12]
```

```
result = detect_outliers(test_data, z_threshold=2.5)
```

```
print('IQR Fences:', result['iqr_fences'])

print('IQR Outliers:', result['iqr_outliers'])

print('Z-score Outliers:', result['z_outliers'])

print('Both methods agree on:', result['agreed_outliers'])
```

Why compare both methods? IQR is robust and non-parametric — it works even with non-normal data. Z-score assumes roughly normal distribution and is sensitive to extreme outliers (a single massive outlier inflates the mean and SD, making other outliers look less extreme). Agreed outliers are the most confidently flagged ones. Disagreements tell you something interesting: maybe the data is skewed, or one method is too aggressive. In practice, I'd always visualise with a box plot after running this function.

Q11

Write Python code to generate a side-by-side Box Plot and Violin Plot comparing the score distributions of three student groups (Group A, B, and C). Add proper labels, colour styling, and annotate the median line for each group.

☒ **Answer**

This is a classic exploratory data analysis task. I'll use matplotlib for the box plot and seaborn for the violin plot, arranged side by side in one figure.

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns

# — Generate sample data —
```

```

np.random.seed(42)

df = pd.DataFrame({
    'Group A': np.random.normal(loc=65, scale=12, size=100), # Average group
    'Group B': np.random.normal(loc=75, scale=8, size=100), # Better, consistent
    'Group C': np.concatenate([ # Bimodal group
        np.random.normal(50, 5, 50),
        np.random.normal(85, 5, 50)
    ])
})

# Reshape to long format for seaborn
df_long = df.melt(var_name='Group', value_name='Score')

# — Create side-by-side figure —
fig, axes = plt.subplots(1, 2, figsize=(14, 6), facecolor='#F8F9FA')

fig.suptitle('Student Score Distribution by Group', fontsize=14,
fontweight='bold', y=1.01)

palette = {'Group A': '#2E75B6', 'Group B': '#1F6B4E', 'Group C': '#C0562A'}

# — Left: Box Plot —
sns.boxplot(ax=axes[0], x='Group', y='Score', data=df_long,
    palette=palette, width=0.5, linewidth=1.5,
    flierprops=dict(marker='o', markersize=4, alpha=0.5))
axes[0].set_title('Box Plot – Five Number Summary', fontweight='bold')
axes[0].set_xlabel('Group')

```

```

axes[0].set_ylabel('Score')

axes[0].grid(axis='y', alpha=0.3)

# Annotate medians on box plot

for i, group in enumerate(['Group A', 'Group B', 'Group C']):

    med = df[group].median()

    axes[0].annotate(f'Median: {med:.1f}',

xy=(i, med), xytext=(i + 0.3, med + 2),

fontsize=8, color='#333333',

arrowprops=dict(arrowstyle='->', color='grey', lw=0.8))

# — Right: Violin Plot —

sns.violinplot(ax=axes[1], x='Group', y='Score', data=df_long,

palette=palette, inner='quartile', linewidth=1.5)

axes[1].set_title('Violin Plot – Full Distribution Shape', fontweight='bold')

axes[1].set_xlabel('Group')

axes[1].set_ylabel('')

axes[1].grid(axis='y', alpha=0.3)

plt.tight_layout()

plt.savefig('/mnt/user-data/outputs/score_distribution_plot.png', dpi=150,

bbox_inches='tight')

plt.show()

print('Plot saved.')

```

Key insight this code reveals: The box plot will show Group C's spread as wide, but the violin plot will

clearly reveal its bimodal shape — two humps indicating two distinct score clusters (50s and 85s). This is exactly the kind of information that a box plot hides but a violin plot exposes. That's why choosing the right visualisation matters.

Q12

Write Python code to generate and visualise a QQ Plot for a dataset of exam scores. Then demonstrate what happens to the QQ plot when the same data is log-transformed — explain the interpretation at each step.

 Answer

QQ plots are essential before running any test that assumes normality (t-tests, ANOVA, linear regression). Let me demonstrate the full workflow.

```
import numpy as np

import scipy.stats as stats

import matplotlib.pyplot as plt

# — Generate right-skewed exam score data —

np.random.seed(7)

raw_scores = np.random.exponential(scale=30, size=200) + 40 # Right-skewed
log_scores = np.log(raw_scores) # Log transformation

# — Create 2x2 figure: histogram + QQ for raw and transformed —

fig, axes = plt.subplots(2, 2, figsize=(12, 8), facecolor='#F8F9FA')

fig.suptitle('QQ Plot Analysis — Raw vs. Log-Transformed Scores', fontsize=13,
fontweight='bold')

# Row 1: Raw scores
```

```
axes[0, 0].hist(raw_scores, bins='fd', color='#2E75B6', edgecolor='white',  
alpha=0.8)
```

```
axes[0, 0].set_title('Raw Scores - Histogram')
```

```
axes[0, 0].set_xlabel('Score'); axes[0, 0].set_ylabel('Frequency')
```

```
stats.probplot(raw_scores, dist='norm', plot=axes[0, 1])
```

```
axes[0, 1].set_title('Raw Scores - QQ Plot')
```

```
axes[0, 1].get_lines()[1].set_color('red') # Reference line in red
```

```
# Row 2: Log-transformed scores
```

```
axes[1, 0].hist(log_scores, bins='fd', color='#1F6B4E', edgecolor='white',  
alpha=0.8)
```

```
axes[1, 0].set_title('Log-Transformed Scores - Histogram')
```

```
axes[1, 0].set_xlabel('log(Score)'); axes[1, 0].set_ylabel('Frequency')
```

```
stats.probplot(log_scores, dist='norm', plot=axes[1, 1])
```

```
axes[1, 1].set_title('Log-Transformed Scores - QQ Plot')
```

```
axes[1, 1].get_lines()[1].set_color('red')
```

```
plt.tight_layout()
```

```
plt.savefig('/mnt/user-data/outputs/qq_plot_analysis.png', dpi=150,  
bbox_inches='tight')
```

```
plt.show()
```

```
# — Print skewness before/after —
```

```
from scipy.stats import skew
```

```
print(f'Raw skewness: {skew(raw_scores):.3f} → highly right-skewed')
```

```
print(f'Log skewness: {skew(log_scores):.3f} → near-normal after transformation')
```

How to read a QQ plot: If points follow the diagonal reference line → data is approximately normal. If points curve upward at both ends (S-shape) → heavy tails. If points curve like a 'J' → right skew. After log transformation, the QQ plot should show points much closer to the diagonal. This is the visual confirmation that the transformation worked and you can now safely apply parametric tests.

Section D — Database Scenario-Based Coding Questions (3 Questions)

Q13

DATABASE SCENARIO: You have a table called `employee_salaries` with columns: `employee_id`, `department`, `salary`, and `hire_year`. Write SQL queries to compute the Mean, Median (approximate), Mode, and Standard Deviation of salaries — both overall and grouped by department.

SQL Answer

SQL is a bit more limited than Python for statistics, but modern databases have powerful window functions and aggregations. Let me break this into parts.

```
-- _____
```

```
-- PART 1: Overall Summary Statistics
```

```
-- _____
```

```
SELECT
```

```
  COUNT(*) AS total_employees,
```

```
  ROUND(AVG(salary), 2) AS mean_salary,
```

```

ROUND(STDDEV(salary), 2) AS std_dev_salary,
MIN(salary) AS min_salary,
MAX(salary) AS max_salary,
MAX(salary) - MIN(salary) AS range_salary,
ROUND(PERCENTILE_CONT(0.25)
WITHIN GROUP (ORDER BY salary), 2) AS q1_salary,
ROUND(PERCENTILE_CONT(0.50)
WITHIN GROUP (ORDER BY salary), 2) AS median_salary, -- Exact median
ROUND(PERCENTILE_CONT(0.75)
WITHIN GROUP (ORDER BY salary), 2) AS q3_salary
FROM employee_salaries;

```

```
-- _____
```

```
-- PART 2: Mode of salary (most frequently occurring value)
```

```
-- _____
```

```

SELECT salary AS mode_salary, COUNT(*) AS frequency
FROM employee_salaries
GROUP BY salary
ORDER BY frequency DESC
LIMIT 1;

```

```
-- _____
```

```
-- PART 3: Department-wise summary with percentiles
```

```
--
SELECT
    department,
    COUNT(*) AS headcount,
    ROUND(AVG(salary), 2) AS mean_salary,
    ROUND(PERCENTILE_CONT(0.50)
    WITHIN GROUP (ORDER BY salary), 2) AS median_salary,
    ROUND(STDDEV(salary), 2) AS std_dev,
    ROUND(STDDEV(salary) / AVG(salary) * 100, 2) AS cv_percent, -- Coefficient of
    Variation
    ROUND(PERCENTILE_CONT(0.25)
    WITHIN GROUP (ORDER BY salary), 2) AS q1,
    ROUND(PERCENTILE_CONT(0.75)
    WITHIN GROUP (ORDER BY salary), 2) AS q3
FROM employee_salaries
GROUP BY department
ORDER BY mean_salary DESC;
```

Key notes: PERCENTILE_CONT gives the exact continuous median (interpolates between values). PERCENTILE_DISC gives the nearest actual data value. I always prefer PERCENTILE_CONT for salary analysis. The Coefficient of Variation ($CV = SD/Mean \times 100\%$) in the department query lets you compare which departments have the most unequal pay — very useful for HR analysis. Note: PERCENTILE_CONT syntax is standard in PostgreSQL and modern MySQL/MariaDB. In older MySQL, you'd need a workaround with row numbering.

customer_id, product_category, customer_segment ('Premium', 'Standard', 'Basic'), and order_total. Write SQL to build a Contingency Table of customer_segment vs. product_category, include row/column totals, and identify which segment-category combination has the highest purchase frequency.

✓ SQL Answer

Contingency tables in SQL use conditional aggregation — a powerful and often underused technique. Let me build this step by step.

```
-- _____  
  
-- PART 1: Full Contingency Table – Segment vs Category  
  
-- Using conditional aggregation (works in all major databases)  
  
-- _____  
  
SELECT  
  
    customer_segment,  
  
    COUNT(CASE WHEN product_category = 'Electronics' THEN 1 END) AS electronics,  
    COUNT(CASE WHEN product_category = 'Clothing' THEN 1 END) AS clothing,  
    COUNT(CASE WHEN product_category = 'Groceries' THEN 1 END) AS groceries,  
    COUNT(CASE WHEN product_category = 'Books' THEN 1 END) AS books,  
  
    COUNT(*) AS row_total  
  
FROM customer_orders  
  
GROUP BY customer_segment  
  
  
  
UNION ALL  
  
  
  
-- Column totals row  
  
SELECT
```

```

'TOTAL' AS customer_segment,
COUNT(CASE WHEN product_category = 'Electronics' THEN 1 END),
COUNT(CASE WHEN product_category = 'Clothing' THEN 1 END),
COUNT(CASE WHEN product_category = 'Groceries' THEN 1 END),
COUNT(CASE WHEN product_category = 'Books' THEN 1 END),
COUNT(*)
FROM customer_orders
ORDER BY CASE WHEN customer_segment = 'TOTAL' THEN 1 ELSE 0 END,
customer_segment;

-- _____

-- PART 2: Relative Frequency (proportions within each segment)

-- _____

SELECT
customer_segment,
product_category,
COUNT(*) AS freq,
ROUND(COUNT(*) * 100.0 / SUM(COUNT(*))
OVER (PARTITION BY customer_segment), 2) AS pct_within_segment,
ROUND(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER (), 2) AS pct_of_total
FROM customer_orders
GROUP BY customer_segment, product_category
ORDER BY customer_segment, freq DESC;

```

```
-- _____

-- PART 3: Top segment-category combination by volume

-- _____

SELECT customer_segment, product_category, COUNT(*) AS order_count
FROM customer_orders
GROUP BY customer_segment, product_category
ORDER BY order_count DESC
LIMIT 5;
```

This is a real marketing analytics query. The OVER (PARTITION BY customer_segment) window function computes each row's percentage within its segment — you can see 'what fraction of Premium customers buy Electronics?' Part 3 directly answers 'which combination has the highest frequency'. In practice, this analysis feeds Chi-Square independence testing — if segments and categories are independent, the percentages would be similar across all segments.

Q15

DATABASE SCENARIO: You have a sensor_readings table with columns: sensor_id, timestamp, and temperature_celsius. Write SQL to compute the IQR for each sensor, identify outlier readings using the $1.5 \times \text{IQR}$ fence rule, and return only the outlier rows with their deviation from the median.

☒ SQL Answer

IQR-based outlier detection in SQL requires computing percentiles per group and then filtering — this is a great use case for CTEs (Common Table Expressions) to keep the logic clean.

```
-- _____

-- STEP 1: Compute Q1, Q3, IQR, and fences per sensor

-- _____
```

```

WITH sensor_stats AS (
    SELECT
        sensor_id,
        PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY temperature_celsius) AS q1,
        PERCENTILE_CONT(0.50) WITHIN GROUP (ORDER BY temperature_celsius) AS
        median_temp,
        PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY temperature_celsius) AS q3,
        PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY temperature_celsius)
        - PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY temperature_celsius) AS iqr,
        AVG(temperature_celsius) AS mean_temp,
        COUNT(*) AS reading_count
    FROM sensor_readings
    GROUP BY sensor_id
),

```

```
-- _____
```

```
-- STEP 2: Define IQR fences (lower and upper bounds)
```

```
-- _____
```

```

fences AS (
    SELECT
        sensor_id,
        q1, q3, iqr, median_temp, mean_temp, reading_count,
        q1 - (1.5 * iqr) AS lower_fence,
        q3 + (1.5 * iqr) AS upper_fence
    FROM sensor_stats
)

```

```

-- _____
-- STEP 3: Flag and retrieve outlier rows
-- _____

SELECT
    r.sensor_id,
    r.timestamp,
    ROUND(r.temperature_celsius::NUMERIC, 2) AS temperature,
    ROUND(f.median_temp::NUMERIC, 2) AS median_temp,
    ROUND(f.lower_fence::NUMERIC, 2) AS lower_fence,
    ROUND(f.upper_fence::NUMERIC, 2) AS upper_fence,
    ROUND((r.temperature_celsius - f.median_temp)
        ::NUMERIC, 2) AS deviation_from_median,
    CASE
        WHEN r.temperature_celsius < f.lower_fence THEN 'LOW OUTLIER'
        WHEN r.temperature_celsius > f.upper_fence THEN 'HIGH OUTLIER'
    END AS outlier_type
FROM sensor_readings r
JOIN fences f USING (sensor_id)
WHERE r.temperature_celsius < f.lower_fence
    OR r.temperature_celsius > f.upper_fence
ORDER BY r.sensor_id, ABS(r.temperature_celsius - f.median_temp) DESC;

```

Why CTEs? Breaking the logic into sensor_stats → fences → final output makes the query readable and debuggable. Each CTE is a named intermediate result. The deviation_from_median column tells you not just that a reading is an outlier, but how extreme it is — critical for prioritising which sensor alerts

to investigate first. This pattern is widely used in IoT monitoring, manufacturing quality control, and cybersecurity (anomalous network packet sizes).



LEARN SMART. ANALYZE BETTER. GROW FASTER.